

第0章: はじめに — 書籍管理アプリを作ろう！

「プログラミングを学ぶ最良の方法は、実際にものを作ることです。」

このチュートリアルでは、モダン（modern：現代的、最新の、という意味の英単語）なWeb技術を使って、実用的な書籍管理アプリをゼロから構築します。初心者の方でも安心して進められるように、一つひとつ丁寧に解説していきます。

なお、文章の中の **バッククォート（```）** で囲まれた部分`は「コード（プログラムの一部）」を表す印です。プログラミング用語や、コマンド・ファイル名などをこの記号で囲って強調しています。

0. はじめる前に — 「そもそも何？」を全部解説

この章を読み始める前に、まず**プログラミング・Web開発の最も基礎的な用語**を解説します。「もう知ってるよ」という方は読み飛ばして「1. この教材の目的と対象読者」から始めてください。逆に「これから初めてプログラミングをやります」という方は、ここを丁寧に読むと後の章が驚くほどスムーズに進みます。

記号の読み方の予備知識: プログラミングでは見慣れない記号がたくさん出てきます。よく出るものを先にメモしておきましょう。 - **"..."** ダブルクォート（double quote、二重引用符）。文字列を囲む記号。 - **'...'** シングルクォート（single quote、一重引用符）。同じく文字列を囲む記号。 - **`...`** バッククォート（back quote）／テンプレートリテラル。文字列を囲みつつ **`${変数}`** の埋め込みもできる。 - **;** セミコロン（semicolon）。「文（命令）の終わり」を表す印。 - **,** カンマ（comma）。値や項目の区切り。 - **{ }** 中カッコ（波カッコ、curly braces）。コードの「ブロック」やオブジェクトを囲む。 - **[]** 角カッコ（square brackets）。配列（リスト）を囲む。 - **()** 丸カッコ（parentheses）。関数の引数や、計算の優先順位に使う。 - **//** 行コメント。**//** から行末までは説明文として扱われ、プログラムとしては実行されない。 - **`/* ... */`** ブロックコメント。複数行をまとめてコメントにする。

0.1 プログラミングとは何か

プログラミング（programming） とは、コンピュータに「やってほしいこと」を**手順書（プログラム）** として書く作業です。コンピュータは指示された通りにしか動かないので、何をしてほしいかを「コンピュータが理解できる言葉（プログラミング言語）」で書く必要があります。

身近な例で言うと、料理のレシピや、家具の組立説明書に近いものです。「(1)小麦粉を200g計る → (2)ボウルに入れる → (3)水を加える」のように、**順番と条件**を厳密に書くことで、誰がやっても（コンピュータが何度実行しても）同じ結果になるようにします。

覚えておきたい3要素: - プログラム (program) : コンピュータへの指示書そのもの。「ソースコード (source code、元になるコードという意味)」「コード」とも言う。 - プログラミング言語 (programming language) : その指示書を書くための言語。日本語/英語のように種類があり、本書では TypeScript (タイプスクリプト) を使う。 - プログラマー (programmer) /エンジニア (engineer) : プログラムを書く人。

0.2 ソースコード・ファイル・拡張子の超基礎

プログラムは **テキストファイル** (text file : 文字だけが入っているファイル) に書きます。Word文書のような装飾 (太字・色など) を持たない、純粋な文字だけのファイルです。ファイルの末尾には **拡張子** (かくちょうし、extension) という「. (ドット) + 数文字」が付きます。拡張子を見れば「中身がどんな種類のファイルか」が分かるようになっています。

拡張子	何のファイルか	このチュートリアルでの登場場面
<code>.html</code>	Webページの構造を書くファイル	結果として生成されるが、自分ではあまり書かない
<code>.css</code>	見た目 (色・サイズ・配置) を指定するファイル	第9章のスタイリングで登場
<code>.js</code>	JavaScript (ブラウザで動くプログラミング言語) を書くファイル	あまり直接書かない
<code>.ts</code>	TypeScript (JavaScriptに型を足した言語) を書くファイル	第2章以降ずっと使う
<code>.tsx</code>	TypeScript + JSX (HTMLっぽい記法) を書くファイル	Reactのコンポーネントで使う
<code>.json</code>	設定情報やデータをやり取りする形式のファイル	<code>package.json</code> などで頻出
<code>.md</code>	Markdown (このチュートリアル自身の形式)。説明文用	教材ファイルの拡張子

▼ 例: `App.tsx` という名前のファイルは、「中身が TypeScript + JSX で書かれた、`App` という名前のファイル」を意味します。ファイル名 (`App`) と拡張子 (`.tsx`) は「.」(ドット) で区切られています。慣習的にコンポーネントを書くファイルは先頭を大文字 (`App`) にすることが多いです。

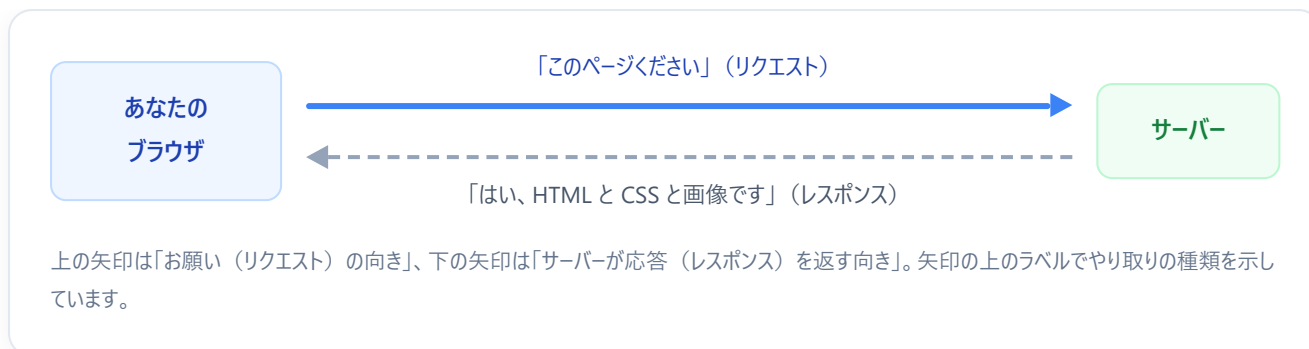
0.3 Webブラウザとサーバーの関係

あなたが今このページを見ているソフトを **ブラウザ** (browser、ブラウズ = 閲覧するもの) と呼びます。Chrome (クローム)、Edge (エッジ)、Safari (サファリ)、Firefox (ファイヤーフォックス) などです。

ブラウザの仕事は大きく2つあります。

1. インターネット上のどこかにあるサーバーから、HTML・CSS・JavaScript などのファイルを取ってくる
2. 取ってきたファイルを解釈して、画面に表示する／プログラムを実行する

サーバー (server、サーブ = 提供する者) とは、24時間ずっと電源が入っていて「リクエストが来たら何かを返す」役割のコンピュータのことです。Amazonの倉庫の係員のように「注文 (リクエスト)」を受けて「商品 (レスポンス)」を返します。



▼ 実例: ブラウザのアドレスバーに `https://example.com/about` と打つと、

1. ブラウザは `example.com` というサーバーを探しに行く (DNS という仕組みで住所を解決。DNS = Domain Name System、ドメイン名をIPアドレスに変換する仕組み)
2. そのサーバーに「`/about` のページが欲しい」とリクエストする (`/about` の部分を「パス」と呼ぶ。サイト内のどのページかを示す)
3. サーバーは HTML を返す
4. ブラウザはその HTML を読み解いて画面を描画する

このやり取りを HTTP (HyperText Transfer Protocol、ハイパーテキスト転送規約) 通信 と呼びます。`https://` の `s` は Secure (セキュア、暗号化されている) の意味です。URL (ユールエル、Uniform Resource Locator) の構造は次のようになっています。

```
https :// example.com : 443 / about ? q=book #section1
|         |         |         |         |         |
|         |         |         |         |         └─ フラグメント (ページ内の位置。先頭が #)
|         |         |         |         └─ クエリ文字列 (追加情報。先頭が ?)
|         |         └─ パス (サイト内のどのページか)
|         └─ ポート番号 (省略可。https の既定値は 443)
└─ ホスト名 / ドメイン (どのサーバーか)
└─ スキーム (通信の種類。http や https)
```

0.4 フロントエンドとバックエンドの違い

Web開発の現場ではよく「フロントエンド／バックエンド／フルスタック」という言葉を使います。「フロント (front)」は前、「バック (back)」は後ろ、「エンド (end)」は端という意味で、ユーザーから見える「前側」とサーバーで動いている「後ろ側」を表しています。

用語	意味	担当する技術（本書で使うもの）
フロントエンド（frontend）	画面側、ユーザーが直接操作する部分	HTML / CSS / TypeScript / React / Next.js
バックエンド（backend）	サーバー側、データの保存や認証などを担当	Supabase（自動でAPIを生成）
データベース（database）	データを保存しておく場所	PostgreSQL（Supabase内蔵）
フルスタック（fullstack）	フロントもバックも両方できる人/構成	本書はこの構成を目指す

本書のポイント: 通常はバックエンドを自分でゼロから作る必要がありますが、**Supabase（スーパベース）** がそれを肩代わりしてくれます。だから「フロントエンドだけに集中していれば、結果としてフルスタックなアプリが完成する」というお得な構成になっています。

0.5 コードを書く・動かすための道具

このチュートリアルで実際に使う道具（ツール）の名前を、先に頭出ししておきます。詳しいインストール方法は第1章で説明します。

道具	役割	例えるなら
VS Code	コードを書くためのエディタ（高機能なメモ帳）	原稿用紙 + 校閲機能
ターミナル（コマンドプロンプト/PowerShell/Terminal）	キーボードから命令文を打ってPCを操作する画面	レストランでの口頭注文
Node.js	JavaScript/TypeScript をブラウザ外でも動かせるようにする実行環境	家庭用キッチン
npm（エヌピーエム）	他人が作った便利なプログラム部品（パッケージ）をダウンロードして管理するツール	アマゾン的な部品通販
Git（ギット）	「いつ・誰が・何を変えたか」を記録するバージョン管理ツール	編集履歴付きノート
GitHub（ギットハブ）	Gitで管理したコードをインターネット上に保管する場所	コード版のクラウドストレージ
ブラウザ	作ったWebアプリを表示・確認するソフト	完成品の試食皿

ターミナル画面の「プロンプト記号」について: ターミナルを開くと、命令を待ち受ける目印（プロンプト、prompt）として `$`、`>`、`%`、`#` などが行頭に出ます。これは「ここから先に命令を打ってね」というだけのマークで、命令の一部ではありません。本書のサンプルでも `$ npm install` のように書かれていたら、`$` は打ち込まずに `npm install` の部分だけを入力します。 - `$` ... macOS/Linux の Bash や Zsh で標準的 - `>` ... Windows のコマンドプロンプトや PowerShell でよく見る - `%` ... macOS の Zsh の既定 - `#` ... 管理者（root）として開いた時の印

0.6 「コードを実行する」ってどういうこと？

このチュートリアルでは何度も「このコードを実行してみましょう」と書きます。「実行」とは、書いたプログラムをコンピュータに読み取らせて、その通りに動かすことです。本書では2通りの実行方法が登場します。

(A) ターミナルでコマンドを打って実行する

ターミナルを開いて、たとえば次のように入力します。

```
node hello.js
# ↑ node というコマンドで、hello.js というファイルを実行する命令
# node          : Node.js を起動するコマンド名（実行ファイル）
# （半角スペース） : コマンドと引数の区切り。スペースは必ず半角（全角だとエラー）
# hello.js      : 実行したいファイル名（拡張子も含めて指定する）
# Enter を押すとこの行が実行され、Node.js が hello.js を読み込んで処理する
```

これは「`hello.js` というファイルの中身を、Node.js（ノードジェイエス）に読ませて実行してね」という意味です。Enter キーを押すと、画面（ターミナル）に結果が出力されます。

▼ 実行結果の例:

```
こんにちは、世界！
```

「コマンド」とは何?: コマンド（command）＝ 命令。ターミナルに打つ「短い英単語の指示」のこと。`node`、`npm install`、`git status` などはずべてコマンドです。コマンドの後ろにスペースを空けて足す追加情報を「引数（ひきすう、argument）」、ハイフンが付いた指定（`-r` や `--save-dev` など）を「オプション／フラグ（flag）」と呼びます。

(B) ブラウザで開いて実行する

Webアプリの場合、コードはサーバーが動かしたり、ブラウザが動かしたりします。本書では `npm run dev` というコマンドで開発用のサーバーを起動し、ブラウザから `http://localhost:3000` にアクセスして動作を確認します。

▼ 起動コマンドと出力の例:

```
npm run dev
# npm      : Node.js に付属するパッケージマネージャ（部品管理ツール）のコマンド
# run      : 「スクリプト（あらかじめ登録された命令）を実行する」サブコマンド
# dev      : 実行するスクリプト名。慣習的に「開発用サーバーを起動する」用途で使われる
# となると「package.json に書かれている dev スクリプトを動かしてね」という意味
```

```
> next dev

  ▲ Next.js 15.0.0
  - Local:      http://localhost:3000

✓ Ready in 2.1s
```

ブラウザで `http://localhost:3000` を開くと、自分で作ったページが表示されます。`localhost`（ローカルホスト）は「自分自身のPC」を意味する特別な住所、`3000` はポート番号（同じPC内のどの窓口かの番号）です。`http://` と `localhost` の間にある `://` はURLの「スキーム（通信の種類）」と「住所」を区切る決まりの記号です。

0.7 「変数・関数・型」のひと言ミニ辞典

第2章以降で詳しく説明しますが、ここで先取りで意味だけ頭に入れておきましょう。

用語	ひと言で言うと	一行サンプル
変数（variable）	値に名前を付けて入れておく箱	<code>const name = "太郎";</code>
関数（function）	処理に名前を付けてまとめたもの。呼ぶと動く	<code>function greet() { ... }</code>
引数（ひきすう、argument）	関数を呼ぶときに渡す材料	<code>greet("太郎")</code> の <code>"太郎"</code>
戻り値（return value）	関数が処理結果として返す値	<code>return name;</code> の <code>name</code>
型（type）	「ここには文字列だけ入れてね」というルール	<code>name: string</code>
配列（array）	値の並び（リスト）	<code>[1, 2, 3]</code>
オブジェクト（object）	「キーと値」の組をまとめたもの	<code>{ name: "太郎", age: 20 }</code>

▼ 一行ずつの実行結果のイメージ:

```
// const = 「変更できない変数（定数）」を作る宣言キーワード / name = 変数名（自分で付ける箱の名前）  
/ = は「右の値を左に入れる」代入演算子 / "太郎" はダブルクォートで囲まれた文字列（string型） / 末尾  
の ; は「文の終わり」を示すセミコロン  
const name = "太郎";  
// console = ブラウザやNode.jsが用意している「コンソール出力用オブジェクト」 / . は「オブジェクト  
の中の機能と呼ぶ」アクセス演算子 / log は「文字列などをコンソールに表示する関数」 / (name) で先ほど  
作った変数 name を引数として渡している / ; で文を終わる  
console.log(name);  
  
// ▼ 実行結果  
// 太郎
```

ここまでが「読む前の超基礎」です。すべてを覚える必要はなく、「分からない単語が出てきたらこの 0 章に戻ってくる」という辞書のように使ってください。

1. この教材の目的と対象読者

目的

この教材は、以下のことを目的としています。

- モダンなWeb開発の基礎を、手を動かしながら体系的に学ぶ
- TypeScript + React + Next.js + Supabase という実務でも広く使われている技術スタック（technology stack：技術の組み合わせ）を習得する
- 一つのアプリを最初から最後まで作り切ることで、開発の全体像を理解する
- CRUD（クラッド。Create=作成、Read=読み取り、Update=更新、Delete=削除の頭文字で、データ操作の基本4つを指します）操作を通じて、Webアプリ開発の基本パターンを身につける

「CRUD」とは？ ほぼ全てのWebアプリは、データの「作成・読み取り・更新・削除」という4つの操作で成り立っています。SNSなら投稿の作成・表示・編集・削除、ECサイト（Electronic Commerce、ネット通販サイト）なら商品の登録・一覧表示・情報更新・削除です。この4つをマスターすれば、どんなWebアプリでも基本は作れるようになります。

対象読者

この教材は、以下のような方を対象としています。

対象	説明
プログラミング初学者	HTML/CSSの基本は分かるが、本格的なWebアプリを作ったことがない方
他言語からの転向者	PythonやJavaなど他の言語の経験があり、Web開発を始めたい方
フロントエンド入門者	バックエンドの経験はあるが、React等のフロントエンド技術を学びたい方
学生・独学者	ポートフォリオに載せられる作品を作りたい方

安心してください！ 分からないことがあっても、各章で丁寧に説明します。エラーが出ても慌てず、一步步進めていきましょう。

2. Webアプリはどのように動いているのか

書籍管理アプリを作り始める前に、Webアプリがどのような仕組みで動いているのか、全体像を理解しておきましょう。ここで全てを覚える必要はありません。「ふーん、こういう仕組みなんだ」くらいの理解で十分です。

レストランに例えると

Webアプリの仕組みは、レストランに例えるとわかりやすくなります。

レストラン	Webアプリ	説明
メニュー・内装	フロントエンド（Frontend：ユーザーが直接見て触れる画面部分）	お客さんが見る・触れる部分。見た目の美しさや使いやすさが重要
厨房	バックエンド（Backend：サーバー側で動くプログラム。データの処理や保存を担当）	お客さんからは見えないが、注文を処理する裏方。データの処理を担当
食材倉庫	データベース（Database：データを永続的に保存する場所。電源を切ってもデータが消えない）	食材（データ）を保管する場所。必要なときに取り出せる
注文票	API（Application Programming Interface：アプリ同士がデータをやり取りするための窓口・ルール）	厨房と客席をつなぐ「注文の仕組み」。フロントエンドとバックエンドの通信手段
ウェ이터	HTTP通信（HyperText Transfer Protocol：ブラウザとサーバー間でデータをやり取りする通信の決まりごと）	注文を厨房に届け、料理を客席に運ぶ人。データの運び役

Webアプリの動作の流れ

例えば、あなたがブラウザ（Chrome や Edge など）で「Amazon」を開いたとき、裏側ではこんなことが起きています。

1. ブラウザがURLを入力 → 「amazon.co.jp の情報をください」とサーバーにリクエスト（要求）を送る
2. サーバーがリクエストを受け取る → 必要なデータをデータベースから取得する
3. サーバーがHTMLを生成 → 商品情報を含んだページのデータを作成する
4. ブラウザがHTMLを受け取る → 受け取ったデータをもとに画面を描画（レンダリング rendering：画面に絵を描く処理）する
5. ユーザーが操作する → 例えば「カートに追加」ボタンを押すと、再びサーバーにリクエストが送られる

このチュートリアルでは：フロントエンド（画面）は **React + Next.js** で作り、バックエンド（データの処理・保存）は **Supabase** というサービスに任せます。Supabase を使うことで、バックエンドのプログラムを自分で書く必要がなくなり、画面の開発に集中できます。

クライアントとサーバー

Webの世界では、「お願いする側」と「応える側」の2つの役割があります。

- **クライアント**（Client：サービスを利用する側）＝ あなたのブラウザ。「このページを見せて」「このデータを保存して」とお願いする側
- **サーバー**（Server：サービスを提供する側）＝ インターネット上のコンピュータ。お願いに応じてデータを返す側

この「クライアントがリクエストを送り、サーバーがレスポンス（応答）を返す」というやり取りが、Webアプリの基本です。

3. 完成イメージ

このチュートリアルで作成する「書籍管理アプリ」は、以下のような画面構成になっています。

 書籍管理アプリ

+ 新規登録

🔍 タイトルや著者名で検索...

リーダブルコード

✓ 読了

著者: Dustin Boswell

編集 削除

プロを目指す人のためのTypeScript入門

📖 読書中

著者: 鈴木 僚太

編集 削除

達人プログラマー

■ 未読

著者: David Thomas

編集 削除

書籍登録・編集画面

書籍管理アプリ > 新規登録

タイトル

書籍のタイトルを入力

著者

著者名を入力

出版年

例: 2024

ステータス

☐ 未読 ☐ 読書中 ☒ 読了

メモ

感想や覚え書きなどを入力...

キャンセル

保存する

主な機能

- **一覧表示:** 登録した書籍をリスト形式で表示
- **新規登録:** タイトル・著者・出版年・ステータス・メモを入力して書籍を登録
- **編集:** 登録済みの書籍情報を変更
- **削除:** 不要な書籍データを削除
- **検索:** タイトルや著者名で書籍を絞り込み

3. 学習ロードマップ

以下のステップで、段階的にアプリを完成させていきます。焦らず、一つずつ進めていきましょう。「ロードマップ（roadmap）」は「道筋を示した地図」という意味の英単語です。



第0章: はじめに（この章）

全体概要・技術スタック・学習ロードマップ



第1章: 開発環境の構築

Node.js / VS Code / Git



第2章: TypeScript入門

型の基本を学ぶ



第3章: React入門

コンポーネントの考え方



第4章: Next.jsプロジェクト作成

プロジェクトの雛形を作る



第5章: UIの構築

画面のレイアウトを作る



第6章: Supabaseのセットアップ

データベースを準備する



第7章: CRUD操作の実装

データの作成・読取・更新・削除



第8章: 検索・フィルタ機能

使いやすさを向上させる





第9章: デプロイ

アプリを公開する



完成！

書籍管理アプリの完成です。おめでとうございます！

各章の目安学習時間は **30分～1時間程度** です。全体を通して、**約1～2週間** で完走できる構成になっています。

4. 使用技術スタック一覧

このチュートリアルで使用する技術の一覧と、それぞれの役割をまとめます。「スタック（stack）」は「積み重ね」という意味で、複数の技術を組み合わせて使うことを指します。

技術	バージョン目安	カテゴリ	役割
TypeScript	5.x	プログラミング言語	JavaScript（Webブラウザで動くプログラミング言語）に「型」（データの種類の指定）を追加した言語。コードの安全性と可読性を向上させる
React	19.x	UIライブラリ（User Interface Library：画面の部品を作るための道具箱）	ユーザーインターフェース（画面）を効率的に構築するためのライブラリ。Meta社（旧Facebook）が開発
Next.js	15.x	Webフレームワーク（Framework：アプリ開発の骨組みを提供するソフトウェア）	Reactベースのフレームワーク。ルーティング（URLに応じたページの切り替え）やサーバーサイド機能を提供する。Vercel社が開発
Supabase	-	BaaS（Backend as a Service：バックエンド機能をクラウドで提供するサービス）	データベース・認証・APIを提供するサービス。バックエンドを自前で構築する手間を省ける。オープンソース
Node.js	20.x 以上	ランタイム（Runtime：プログラムを実行する環境）	JavaScriptをブラウザの外（自分のPC上）で実行するための環境。開発ツールの動作に必要
npm	10.x	パッケージマネージャ（Package Manager：ライブラリの管理ツール）	他の開発者が作った便利なプログラム（パッケージ）をインストール・管理するツール
Git	2.x	バージョン管理システム（VCS：コードの変更履歴を記録する仕組み）	コードの変更履歴を記録・管理するツール。「いつ、誰が、何を变えたか」を追跡できる
VS Code	最新版	コードエディタ（IDE：統合開発環境に近い高機能エディタ）	コードを書くためのエディタ。Microsoft社が開発。豊富な拡張機能が利用可能

「バージョン」って何？ ソフトウェアは改良されるたびに番号が振り直されます。「5.x」と書いてあれば「メジャー番号が5の系列ならどの細かい版でも OK」という意味です。x の部分は「何でもいい」を表すワイルドカードです。表記は SemVer（セマンティックバージョンing、Semantic Versioning）と呼ばれ、メジャー・マイナー・パッチ の3階層が標準です。

各技術の概要

TypeScript — 「型」のある JavaScript

TypeScriptは、JavaScriptに静的型付け（Static Typing：コードを実行する前に、変数や関数に入るデータの種類をチェックする仕組み）の機能を追加した言語です。

「この変数には文字列しか入らない」「この関数は数値を返す」といったルールをコードに書けるため、間違いを事前に防ぐことができます。

身近な例え：型は「ラベル付きの箱」のようなものです。「みかん専用」と書いてある箱にりんごを入れようすると、入れる前に「それはみかんじゃないよ！」と教えてくれます。プログラムでも同じように、間違ったデータを入れようすると、コードを書いている段階でエラーが表示されます。

```
// =====
// 比較サンプル: 同じ「足し算」を JS と TS で書いた場合
// -----
// この例は「TypeScriptがいきなり早い段階でバグを教えてくれるか」を示すための比較。
// =====
// で始まる行は「コメント」と呼ばれ、プログラムとしては実行されない。
// 上の行のように
// 自分や読み手向けの説明文として自由に書ける。

// ----- JavaScript の場合 (型なし) -----
// function は「関数を定義する」キーワード (予約語)
// add は関数名。自分で好きに名付けられる (小文字始まりが慣習)
// (a, b) は引数リスト。a と b の2つの値を受け取る、と宣言している
// 型注釈が無いので a も b も「どんな種類の値でもOK」 (= 動的型付け)
function add(a, b) {
  // return は「関数の結果を呼び出し元へ返す」キーワード
  // a + b: + 演算子は「両方が数値なら算術加算」「片方でも文字列なら文字列連結」になる
  return a + b;
  // 末尾の ; (セミコロン) は「この文はここで終わり」を示す
}

// (1) "1" は文字列リテラル (ダブルクォートで囲むと文字列扱い)、2 は数値リテラル
// JavaScriptは「文字列 + 数値」を「文字列の連結」として処理する。
// よって "1" + 2 → "12" という文字列が返ってくる。
// add 関数を呼び出している。( ) の中が引数。複数あればカンマ , で区切る
add("1", 2);
// ▼ 実行結果 (変数に入れて console.log すると)
// "12" ← 数字の12ではなく文字列の "12"。気づきにくいバグ!

// ----- TypeScript の場合 (型あり) -----
// 引数の後ろに「: 型名」を書くと、その引数は指定した型しか受け付けなくなる
// a: number → a は number (数値) 型のみ
// b: number → b も number 型のみ
// ) の後ろの「: number」は「この関数の戻り値の型」を指定している (戻り値も数値)
function add(a: number, b: number): number {
  // a も b も数値だけなので、ここでは必ず数値の加算になる
  return a + b;
}

// (2) "1" は文字列なので、a: number に渡すと型の不一致になりエラー
add("1", 2);
// ▼ コンパイル時のエラー (VS Codeでは赤い波線で即表示。「コンパイル」=実行前の変換処理)
// Argument of type 'string' is not assignable to parameter of type 'number'.
// ('string' 型の引数は 'number' 型のパラメータには代入できません)
//
// → 実行する前にミスに気づける! これが TypeScript の最大の価値。
```

React — コンポーネントで画面を構築

Reactは、画面を**コンポーネント**（Component：UIの部品。ボタン、カード、フォームなど、画面の構成要素一つひとつ）という小さな部品に分割して構築するライブラリ（Library：特定の機能を提供するプログラムの集まり）です。

身近な例え：レゴブロックを想像してください。小さなブロック（コンポーネント）を組み合わせて、家や車（ページ全体）を作ります。一度作ったブロックは、別の作品でも再利用できます。Reactも同じで、「書籍カード」というコンポーネントを一度作れば、一覧画面で何回でも使い回せます。

Next.js — Reactをさらに便利にするフレームワーク

Next.jsは、Reactだけでは面倒な**ページ遷移の管理**（ルーティング：URLに応じて表示するページを切り替える仕組み）や**サーバー側での処理**（SSR：Server Side Rendering、サーバー上でHTMLを生成してからブラウザに送る方式）を簡単に実現できるフレームワーク（Framework：アプリ開発の骨組みや決まりごとを提供するソフトウェア）です。

Reactだけだと何が困るの？ Reactは画面の「部品」を作ることに特化したライブラリなので、「URLを変えたらページを切り替える」「検索エンジンに見つけてもらう」「サーバーでデータを取得する」といった機能は自分で用意する必要があります。Next.jsはこれらを最初から備えており、ファイルを配置するだけでページが作れる仕組みが特徴的です。

Supabase — バックエンドを手軽に構築

Supabaseは、データベース（データの保管庫）やユーザー認証（ログイン機能。認証＝authentication、本人確認）などの**バックエンド機能**をクラウド（インターネット上のサーバー）で提供するサービスです。自分でサーバーを構築する必要がなく、設定するだけですぐにデータの保存・取得ができます。

自分でバックエンドを作るとどうなるの？ サーバーの設定、APIの設計、データベースの構築、セキュリティ対策...と、やることが一気に増えます。Supabaseを使えば、これらを全てクラウド上で数クリックで用意でき、フロントエンド（画面）の開発に集中できます。学習中は特にこれが重要です。

5. なぜこの技術スタックを選んだのか

技術にはたくさんの選択肢があります。ここでは、今回の技術スタックを選んだ理由を、他の選択肢と比較しながら説明します。

UIライブラリの比較: React vs Vue vs Svelte

観点	React	Vue	Svelte
学習コスト	やや高め	低め	低め
求人・需要	非常に多い	多い	少なめ
エコシステム	非常に豊富	豊富	成長中
コミュニティ	世界最大級	大きい（特にアジア圏）	成長中
TypeScriptサポート	良好	良好	良好
大規模開発への適性	高い	高い	発展途上

「エコシステム」とは？ ある技術を中心にした関連ツールや拡張機能、書籍、コミュニティ全体のこと。豊かなエコシステム＝「困った時に解決策が見つかりやすい」という意味でもあります。

Reactを選んだ理由: 最も広く使われているUIライブラリであり、就職・転職活動でも求められることが多いため、学ぶ価値が非常に高いです。エコシステムが充実しているので、困ったときに情報を見つけやすいのも大きなメリットです。

フレームワークの比較: Next.js vs Vite vs Remix

観点	Next.js	Vite + React	Remix
セットアップの簡単さ	簡単	簡単	やや手間がかかる
サーバーサイド機能	充実（SSR/SSG/ISR）	なし（SPA中心）	充実
ファイルベースルーティング	あり	なし（要追加設定）	あり
デプロイのしやすさ	非常に簡単（Vercel）	簡単	やや手間がかかる
学習リソースの豊富さ	非常に多い	多い	少なめ
企業での採用実績	非常に多い	多い	増加中

SSR/SSG/ISR/SPA の用語整理: - SSR (Server Side Rendering)：サーバーでHTMLを作ってからブラウザに送る方式。 - SSG (Static Site Generation)：ビルド時にHTMLを作っておく方式。表示が速い。 - ISR (Incremental Static Regeneration)：SSG をベースに一定間隔でHTMLを再生成する方式。 - SPA (Single Page Application)：1枚のHTMLでブラウザ側がページを切り替える方式。

Next.jsを選んだ理由: ファイルを作るだけでページが追加できる仕組みや、Vercelへのワンクリックデプロイなど、初心者にとって「余計なことを考えずに済む」設計が魅力です。実務での採用例も非常に多く、学んだことがそのまま活かれます。

言語の比較: TypeScript vs JavaScript

観点	TypeScript	JavaScript
型安全性	あり（コンパイル時にエラー検出）	なし（実行時にエラー発覚）
コードの可読性	高い（型が仕様書の役割を果たす）	普通
学習コスト	やや高め（型の概念の学習が必要）	低め
開発効率	高い（エディタの補完が強力）	普通
業界のトレンド	主流になりつつある	依然として広く利用
エラーの発見しやすさ	書いている途中で気づける	実行してから気づく

TypeScriptを選んだ理由: 最初は少し学ぶことが増えますが、型があることでエディタが強力に補助してくれるため、結果的に**初心者**にもやさしい言語です。「何を渡せばいいかわからない」という場面が大幅に減ります。

バックエンドの比較: Supabase vs Firebase vs 自前バックエンド

観点	Supabase	Firebase	自前バックエンド
データベースの種類	PostgreSQL（リレーショナルDB）	Firestore（NoSQL）	自由に選択可能
SQLの学習	できる	できない	できる
無料枠	十分（個人開発に最適）	十分	サーバー費用が必要
セットアップの手間	非常に簡単	簡単	大変（サーバー構築が必要）
学習コスト	低い	低い	高い（API設計等が必要）
オープンソース	はい	いいえ	-
他のサービスへの移行	しやすい（標準SQL）	しにくい（独自仕様）	-

「リレーショナルDB / NoSQL」とは？ - リレーショナルDB（Relational Database, RDB）：表（テーブル）と表の関係でデータを管理する方式。SQLで操作する。PostgreSQL、MySQL など。 - NoSQL（Not Only SQL）：表形式に縛られないデータベースの総称。Firestore は「ドキュメント型」のNoSQL。

Supabaseを選んだ理由: PostgreSQL（業界標準のデータベース）を使っているため、ここで学んだSQL知識は他の環境でもそのまま活かされます。また、オープンソースであり、ダッシュボードが直感的で使いやすく、無料枠も個人学習には十分です。

6. 前提知識

このチュートリアルを始めるにあたって、以下の知識があると学習がスムーズに進みます。

必須の前提知識

- **HTML の基本:** タグ（`<div>` , `<p>` , `<a>` など）の役割が分かること。タグは `<開始タグ>中身</終了タグ>` の形で書く決まりがあります。
- **CSS の基本:** 文字色や背景色の変え方、簡単なレイアウトの作り方が分かること。 `color: red;` のように「プロパティ名: 値;」の形で書きます。
- **プログラミングの基本概念:** 変数、条件分岐（if文）、繰り返し（for文）、関数がどういうものか分かること

あると望ましい知識（なくても大丈夫です）

- **JavaScript の基本:** 変数宣言（`const` / `let`）、アロー関数（`() => { ... }` の形の短い関数の書き方）、配列操作（`map` , `filter` : 配列を加工・抽出する関数）
- **コマンドライン操作:** ターミナル（コマンドプロンプト）で `cd`（change directory : 移動先のフォルダを切り替える）や `ls`（list : 今いるフォルダの中身一覧。Windows では `dir`）が使えること
- **Git の基本:** `git add`（変更を「コミット予定リスト」に登録）、`git commit`（コミット予定をスナップショットとして記録）の意味が分かること

前提知識に不安がある方へ: 心配しなくても大丈夫です。必要な知識はその都度、補足説明を入れています。「まったくの未経験」でなければ、十分についてこれる内容です。分からない部分があったら、飛ばさずにゆっくり読み返してみてください。

7. この教材で学べること

チュートリアルを完走すると、以下のスキルが身につきます。一つずつチェックを付けながら進めていきましょう！（Markdown の `- [ ]` は「未完了のチェックボックス」を表す書き方です）

開発環境

- ☐ Node.js と npm のインストールと基本操作
- ☐ VS Code のセットアップと便利な拡張機能の導入
- ☐ Git によるバージョン管理の基本

TypeScript

- ☐ 基本的な型（`string` : 文字列, `number` : 数値, `boolean` : 真偽値=true/false）の使い方

- [] 型定義（`type` / `interface`：複雑な型に名前を付ける2つの方法）の作成
- [] 関数の引数と戻り値に型を付ける方法

React

- [] コンポーネントの概念と作成方法
- [] JSX（JavaScript XML：JS/TSの中にHTMLっぽい記法を直接書ける構文）の書き方
- [] `useState`（ステートを保持するためのフック関数）を使った状態管理
- [] `useEffect`（副作用を扱うためのフック関数）を使った副作用の処理
- [] Props（プロパティ：親コンポーネントから子コンポーネントへ渡すデータ）によるデータの受け渡し
- [] イベントハンドリング（クリック・入力など：ユーザー操作を受け取って反応する処理）
- [] フォームの作成とバリデーション（validation：入力値の妥当性チェック）

Next.js

- [] プロジェクトの作成と構成の理解
- [] App Router によるファイルベースルーティング（ファイル/フォルダ構成がそのままURLになる仕組み）
- [] Server Components と Client Components の違い
- [] レイアウトの共通化

Supabase

- [] プロジェクトの作成とテーブル設計
- [] データの作成（INSERT：SQLでデータを新規追加する命令）
- [] データの読み取り（SELECT：SQLでデータを取り出す命令）
- [] データの更新（UPDATE：SQLでデータを書き換える命令）
- [] データの削除（DELETE：SQLでデータを消す命令）
- [] フィルタ・検索機能の実装

総合スキル

- [] モダンなWebアプリの設計パターンの理解
- [] CRUD操作の一連の流れの実装
- [] アプリケーションのデプロイ（公開）

8. アプリのアーキテクチャ（全体構成図）

最終的に完成するアプリの全体像を、以下の図で確認しましょう。今は全てを理解する必要はありません。チュートリアルを進めるうちに、それぞれの役割が自然と分かるようになります。「アーキテクチャ（architecture）」は「建築物の構造／設

計」という意味で、ソフトウェアでは「全体の構成」を指します。



books テーブル

```
id title author
published_year
status memo created_at
```

「books テーブル」の各カラム (列) の意味: - **id** ... レコード (行) を一意に識別するための番号。主キー (Primary Key)。 - **title** ... 書籍タイトル (文字列)。 - **author** ... 著者名 (文字列)。 - **published_year** ... 出版年 (数値)。 - **status** ... 読書状態。「未読／読書中／読了」のいずれか。 - **memo** ... 自由記述メモ (文字列、長め)。 - **created_at** ... レコードが作成された日時。自動で入る。

データの流れ（具体例: 書籍を新規登録する場合）



流れの読み方: ステップ1～5は「ユーザーから DB へ向かう登録リクエスト」、ステップ6～10は「DB から画面へ戻ってくる応答」と覚えると分かりやすいです。`POST` はHTTPメソッドの一種で「新規作成」を意味します。`INSERT INTO books ...` はSQL（データベース操作言語）の構文で「books テーブルに行を追加」する命令です。

まとめ

この章では、以下のことを確認しました。

- このチュートリアルで作る **書籍管理アプリ** の完成イメージ
- **学習ロードマップ** と各章の概要
- 使用する **技術スタック** とそれぞれの役割
- なぜこの技術スタックを **選んだのか** の理由
- チュートリアルを始めるための **前提知識**
- この教材で **学べること** の全体像
- アプリの **アーキテクチャ**（全体構成）

準備はできましたか？ 次の章では、実際に開発環境を構築していきます。Node.jsやVS Codeのインストールから始めましょう！

9. この教材を進めるにあたってのアドバイス

プログラミング学習を効果的に進めるためのアドバイスをまとめました。

エラーが出ても慌てない

プログラミングでは、エラーは**日常的に起こるもの**です。プロのエンジニアでも、毎日のようにエラーに遭遇しています。エラーメッセージは「ここが間違っているよ」と教えてくれるヒントなので、むしろ**ありがたい存在**です。

エラーメッセージの読み方のコツ： エラーメッセージは英語で表示されることがほとんどですが、慌てず以下の手順で対処しましょう。1. エラーメッセージの**最後の行**を読む（最も重要な情報が書かれていることが多い） 2. メッセージをそのまま**Google** で検索する（同じエラーに遭遇した人の解決策が見つかります） 3. このチュートリアル**のトラブルシューティング**（trouble shooting：問題解決）セクションを確認する

写経（しゃきょう）から始めよう

最初はコードの意味が完全に理解できなくても、**まずはサンプルコードをそのまま書き写してみよう**（これを「写経」と呼びます）。手を動かすことで、自然と「こういうパターンなんだな」と体が覚えていきます。

分からない箇所は飛ばしてOK

全てを完璧に理解してから次に進む必要はありません。「今は分からないけど、後で分かるようになるかも」と思って、**とりあえず先に進む**のも大切な学習戦略です。後の章で実際にコードを書いたときに、「ああ、あのとき説明されていたのはこういうことか！」と腑に落ちることがよくあります。

各章の目安時間

章	テーマ	目安時間
第0章	はじめに（この章）	15分
第1章	開発環境の構築	30～60分
第2章	TypeScript入門	45～90分
第3章	React入門	60～90分
第4章	Next.js入門	45～90分
第5章	Supabase設定・DB設計	30～60分
第6章	プロジェクト作成	30～45分
第7章	一覧・登録機能	60～90分
第8章	編集・削除・検索	60～90分
第9章	スタイリング・UI改善	45～60分
第10章	デプロイ	30～45分

全体を通して約**1～2週間**（1日1～2時間のペース）で完走できる構成です。もちろん、もっとゆっくりでもまったく問題ありません。

10. よくある質問（FAQ）

FAQ は Frequently Asked Questions（よく聞かれる質問）の略です。

Q: プログラミング完全未経験でも大丈夫ですか？

A: HTML/CSSの基本（タグの書き方や、文字色の変え方など）と、プログラミングの基本概念（変数、if文、for文、関数とは何か）の知識があれば大丈夫です。これらに不安がある場合は、先にProgateやドットインストールなどの入門サービスで基礎を学んでおくとスムーズです。

Q: WindowsとMac、どちらでも進められますか？

A: はい、どちらでも問題なく進められます。第1章の環境構築の手順は、Windows/Mac/Linuxそれぞれに対応しています。

Q: このチュートリアルで作ったアプリは、ポートフォリオ（portfolio：作品集）に使えますか？

A: もちろん使えます。ただし、チュートリアル通りに作っただけではなく、**自分なりのアレンジ**（機能の追加、デザインの変更、別のテーマへの応用など）を加えると、より魅力的なポートフォリオになります。

Q: 有料のサービスは使いますか？お金はかかりますか？

A: このチュートリアルで使用するサービス（Supabase, Vercel, GitHub）はすべて**無料枠**で利用できます。クレジットカードの登録も不要です（Supabaseの無料プランで十分です）。

Q: エラーが解決できないとき、どうすればいいですか？

A: 以下の順序で試してみてください。1. このチュートリアルのトラブルシューティングセクションを確認 2. エラーメッセージをそのままGoogle検索 3. ChatGPTやClaude（AIアシスタント）にエラーメッセージを貼り付けて質問 4. Stack Overflow（プログラマー向けQ&Aサイト）で検索 5. GitHubのIssues（プロジェクトの質問掲示板）で類似の問題を検索

さあ、始めましょう！ 次の章では、開発に必要なツール（Node.js、VS Code、Git）をインストールしていきます。

11. 用語集（Glossary） — 困ったらここへ戻ってくる

このチュートリアルを読んでいて分からない単語に出会ったら、まずこの用語集を見てください。**完璧に覚える必要はありません**。「あ、こういう意味だったな」と思い出せば十分です。Glossary は「用語集」を表す英単語です。

開発環境・ツール系

用語	読み	意味
Node.js	ノードジェイエス	ブラウザ外でJavaScript/TypeScriptを動かす実行環境。 <code>node コマンド</code> で使える
npm	エヌピーエム	Node Package Manager。便利なプログラム部品をダウンロード・管理するツール
パッケージ	ぱっけーじ	他の人が作った再利用可能なプログラムのまとまり
ライブラリ	らいぶらり	特定機能を提供するパッケージ。例：React は UI ライブラリ
フレームワーク	ふれーむわーく	アプリ全体の骨組みを提供する大きめの仕組み。例：Next.js
VS Code	ぶいえずコード	Microsoft製の高機能コードエディタ。無料
ターミナル	たーみなる	キーボードから命令を打って操作する画面（黒い画面）
コマンドプロンプト	こまんどぷろんぷと	Windowsの古めのターミナル
PowerShell	ばわーしえる	WindowsのターミナルNo.2。本書ではこちらを使う
Bash	バッシュ	macOS/Linux で標準的なターミナルのシェル
シェル	しえる	ターミナルでコマンドを解釈する本体
Git	ぎっと	コードのバージョン（変更履歴）管理ツール
GitHub	ぎっとはぶ	Gitで管理したコードを保管する世界最大のサービス
リポジトリ (repo)	りぼじとり	Git/GitHubでコードを保管する単位。プロジェクト1つにつき1つ作るのが普通
コミット	こみっと	Gitで「この時点のスナップショットを記録」する操作
プッシュ	ぷっしゅ	ローカルのコミットをGitHubに送る操作
プル	ぷる	GitHubの最新コミットを自分のPCに取り込む操作

言語・型系

用語	読み	意味
HTML	エイチティーエムエル	文書の構造を書く言語（Hyper Text Markup Language）
CSS	シーエスエス	見た目を指定する言語（Cascading Style Sheets）
JavaScript（JS）	じゃばすくりぷと	ブラウザで動くスクリプト言語
TypeScript（TS）	たいぷすくりぷと	JavaScriptに「型」を追加した言語
型（Type）	かた	「数値」「文字列」など、データの種類のラベル
静的型付け	せいてきかたづけ	コード実行前に型をチェックする方式（TS, Java など）
動的型付け	どうてきかたづけ	コード実行時に型が決まる方式（JS, Python など）
変数	へんすう	値を入れる箱。 <code>const</code> 、 <code>let</code> で宣言する
定数	ていすう	一度入れたら変えられない変数。 <code>const</code> で宣言
関数	かんすう	処理をまとめて名前を付けたもの。 <code>function</code> で定義
引数	ひきすう	関数に渡す入力値
戻り値	もどりち	関数が返す値。 <code>return</code> で指定
配列	はいれつ	値の並び。 <code>[1, 2, 3]</code>
オブジェクト	おぶじえくと	キーと値のペアの集合。 <code>{ name: "太郎" }</code>
アロー関数	あろーかんすう	<code>() => { ... }</code> の形式の関数
インターフェース	いんたーふぇーす	TS で型をまとめて定義する仕組み
ジェネリクス	じぇねりくす	型をパラメータとして受け取る仕組み（ <code><T></code> ）
async/await	エイシンク/アウエイト	非同期処理を書きやすくするキーワード
Promise	ぷろみす	非同期処理の結果を表すオブジェクト

Web/通信系

用語	読み	意味
HTTP	エイチティーティーピー	ブラウザとサーバーの通信規約
HTTPS	エイチティーティーピー エス	HTTPの暗号化版
URL	ユーアールエル	Webページの住所（例： https://example.com/about ）
リクエスト	りくえすと	クライアントからサーバーへの要求
レスポンス	れすぽんす	サーバーからクライアントへの応答
GET	げっと	データの取得に使うHTTPメソッド
POST	ぽすと	データの新規登録に使うHTTPメソッド
PUT/PATCH	ぷっと/ぱっち	データの更新に使うHTTPメソッド
DELETE	でりーと	データの削除に使うHTTPメソッド
ステータスコード	すてーたすこーど	レスポンスの状態を示す3桁の数字。 200=成功 404=見つからない 500=サーバーエラー
API	エーピーアイ	プログラム同士が通信するための窓口
REST API	れすとえーびーあい	URLとHTTPメソッドで操作するAPIの設計スタイル
JSON	じえいそん	データをやり取りする形式。 {"name": "太郎"} のような形
ポート	ぽーと	同じPC内のどの窓口かを示す番号（例：3000）
localhost	ろーかるほすと	「自分自身のPC」を表す住所

React/Next.js 系

用語	読み	意味
コンポーネント	こんぽーねんと	画面を構成する再利用可能な部品
JSX	じえいすえっくす	JS/TSの中にHTMLっぽい記法を埋め込む書き方
Props	ぷろっぷす	コンポーネントの外から渡すデータ
State	すてーと	コンポーネントの内部で持つ変化するデータ
useState	ゆーすすてーと	Reactで状態を持たせるためのフック関数
useEffect	ゆーすえふえくと	Reactで副作用（データ取得など）を行うためのフック
フック	ふっく	<code>use～</code> で始まるReactの関数群（Hooks）
App Router	あっぷるーたー	Next.jsの新しいページ管理方式（フォルダ＝URL）
Server Components	さーばーこんぽーねんと	サーバー側で実行されるコンポーネント（デフォルト）
Client Components	くらいあんとこんぽーねんと	ブラウザ側で実行されるコンポーネント（ <code>"use client"</code> を書く）
SSR	エスエスアール	Server Side Rendering。サーバーでHTMLを生成
CSR	シーエスアール	Client Side Rendering。ブラウザ上でHTMLを生成
SSG	エスエスジー	Static Site Generation。ビルド時にHTMLを生成
ISR	アイエスアール	Incremental Static Regeneration。一定間隔でHTMLを再生成
ハイドレーション	はいどれーしょん	サーバーで作られたHTMLにJSの動きを付ける処理

データベース/Supabase 系

用語	読み	意味
データベース (DB)	でーたべーす	データを永続的に保存する場所
RDB	アールディービー	リレーショナルデータベース。表形式
PostgreSQL	ぽすとぐれすきゅーえる	オープンソースのRDB。Supabaseが内蔵
SQL	エスキューエル	データベースを操作する言語
テーブル	てーぶる	データを保存する表
カラム (列)	からむ	テーブルの縦の項目 (フィールド)
レコード (行)	れこーど	テーブルの横の1件分のデータ
主キー (Primary Key)	しゅきー	レコードを一意に識別するカラム。普通は id
外部キー (Foreign Key)	がいぶきー	他のテーブルの主キーを参照するカラム
CRUD	くらっど	Create / Read / Update / Delete の頭文字。データ操作の基本4つ
Supabase	すーぱべーす	オープンソースのBaaS。PostgreSQL + 認証 + API
RLS	アールエルエス	Row Level Security。行単位の権限制御
環境変数	かんきょうへんすう	設定値を .env ファイルなどに分離して書く仕組み

その他

用語	読み	意味
デプロイ	でぷろい	作ったアプリをサーバーに配置して公開する作業
Vercel	ばーせる	Next.jsを作った会社のホスティングサービス。本書でデプロイ先に使う
ホスティング	ほすていんぐ	作ったアプリをサーバーで動かして公開してくれるサービス
ビルド	びるど	開発用のコードを本番用に変換・最適化する処理
バンドル	ばんどる	多数のJSファイルを1〜数個にまとめる処理
ESLint	いーえすりんと	JS/TSのコードをチェックして悪い書き方を警告するツール
Prettier	ぷりていあー	コードの見た目 (インデントなど) を自動整形するツール

さあ、始めましょう！ 次の章では、開発に必要なツール (Node.js、VS Code、Git) をインストールしていきます。

次の章へ進む → [第1章: 開発環境の構築](#)